

Unidad II

Clase 3

De la abstracción a la implementación

TDA Cola con Prioridad
TDA Conjunto - Implementación

*Comprender que no todas las estructuras
sirven para todos los problemas.*

Llegan pacientes a una guardia

Paciente A – Dolor de cabeza

Paciente B – Fractura expuesta

Paciente C – Fiebre

Paciente D – Paro cardíaco

“Tenemos una cola...”



TDA COLA CON PRIORIDAD



Una cola con prioridad es una estructura de datos en la que cada elemento tiene un valor de "prioridad" asociado, se ordena por llegada, pero los de mayor prioridad se procesan primero

OPERACIONES BÁSICAS

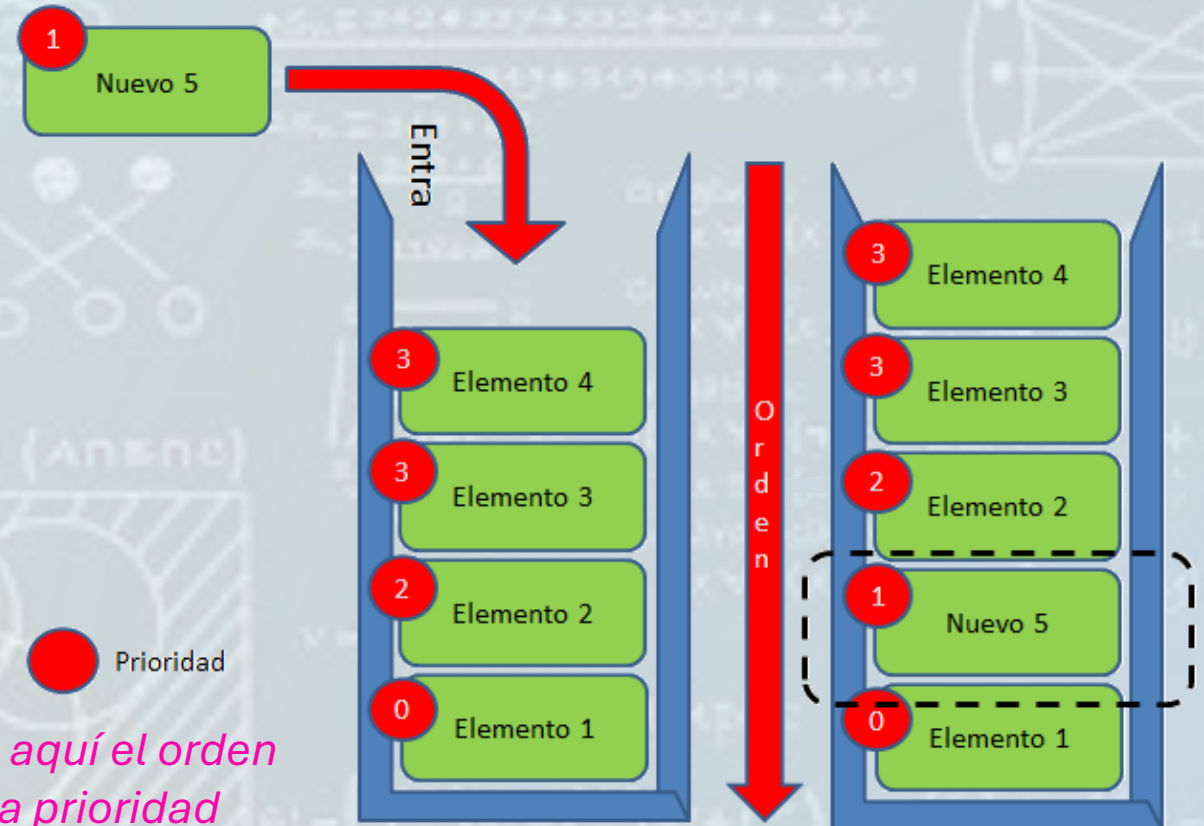
- enqueuePriority(x, p): agregar con prioridad
- dequeue(): quitar el de mayor prioridad
- front(): ver el de mayor prioridad
- priority(): ver la prioridad del primero
- isEmpty(): es vacío
- isFull(): está lleno

PRINCIPIO

- HPF (Highest Priority First)
Primero el de mayor prioridad



A diferencia de una cola normal (FIFO), aquí el orden de salida depende exclusivamente de la prioridad





COLA CON PRIORIDAD

El contrato del TDA determina los **estándares** que constituyen la **representación interna** de la estructura.

REPRESENTACION INTERNA



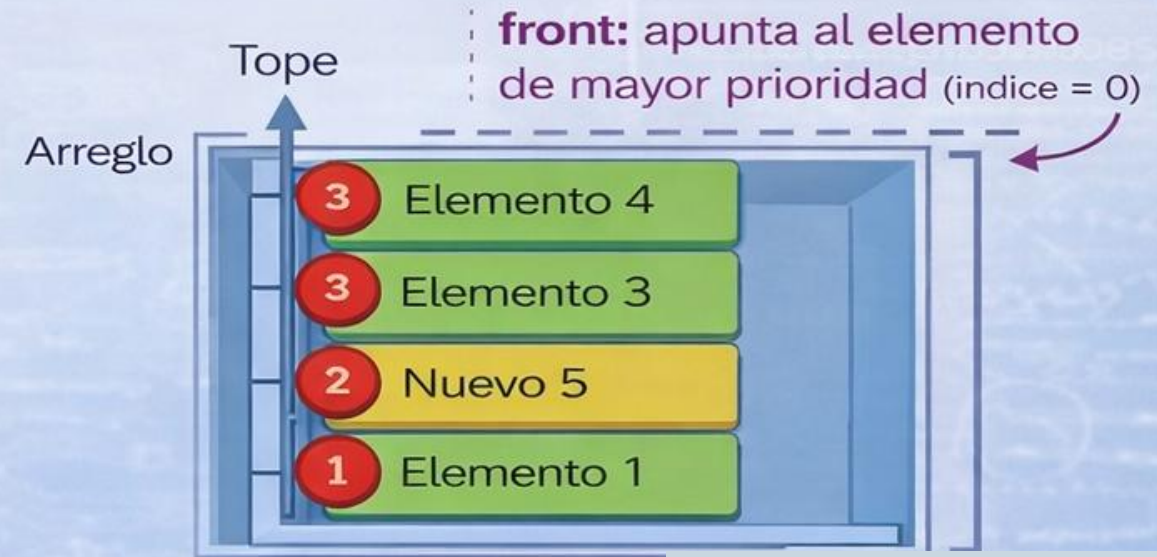
OPERACIONES BÁSICAS

- ✓ **enqueuePriority(x, p)**: agregar con prioridad
- ✓ **dequeue()**: quitar el de mayor prioridad
- ✓ **front()**: ver el de mayor prioridad
- ✓ **priority()**: ver la prioridad del primero
- ✓ **isEmpty()**: es vacío
- ✓ **isFull()**: está lleno

✓ Principio que la define:

- HPF (Highest Priority First)
- Primero el de mayor prioridad

REPRESENTACIÓN INTERNA



Arreglo ordenado de elementos que respetan su prioridad

Principio que la define: **HPF (Highest Priority First)**

Primero el de mayor prioridad





Dos Arreglos

Elementos

A B C D

Prioridades

3 1 4 2

Costo: Inserción

Arreglo de Objetos

A, 3 B, 1 C, 4 D, 2

Costo: Inserción

Múltiples Colas

Cola Alta

A E

Cola Media

C D

Cola Baja

B F

Costo: Extracción



IMPLEMENTACIÓN – Estrategia 1



enqueuePriority(x, p) → void

Pre: la cola no está llena

dequeue() → elemento

Pre: la cola no está vacía

front() → elemento

Pre: la cola no está vacía

priority() → entero

Pre: la cola no está vacía

isEmpty() → boolean

isFull() → boolean

elementos

8	95	7	15	21	8	3	7					
0	1	2	3	4	5	6	7	8	9	10	11	

prioridades

1	3	3	4	5	7	8	8					
0	1	2	3	4	5	6	7	8	9	10	11	

indice

elementos

16

prioridades

5

IMPLEMENTACIÓN – Estrategia 1



```
public class ColaPrioridad {

    private int[] elementos;
    private int[] prioridades;
    private int cantidad;
    private int capacidad;

    public ColaPrioridad(int capacidad) {
        this.capacidad = capacidad;
        elementos = new int[capacidad];
        prioridades = new int[capacidad];
        cantidad = 0;
    }

    public boolean isEmpty() {
        return cantidad == 0;
    }

    public boolean isFull() {
        return cantidad == capacidad;
    }

    // enqueuePriority
    public void acolarPrioridad(int x, int p) {
        if (isFull()) return;

        int i = cantidad - 1;

        // desplazar mientras la prioridad sea menor
        while (i >= 0 && prioridades[i] < p) {
            elementos[i + 1] = elementos[i];
            prioridades[i + 1] = prioridades[i];
            i--;
        }
    }
```

```
        // insertar en la posición correcta
        elementos[i + 1] = x;
        prioridades[i + 1] = p;

        cantidad++;
    }

    // dequeue
    public int desacolar() {
        if (isEmpty()) return -1;

        int valor = elementos[0];

        // desplazar hacia la izquierda
        for (int i = 0; i < cantidad - 1; i++) {
            elementos[i] = elementos[i + 1];
            prioridades[i] = prioridades[i + 1];
        }

        cantidad--;
        return valor;
    }

    // front
    public int primero() {
        if (isEmpty()) return -1;
        return elementos[0];
    }

    public int prioridad() {
        if (isEmpty()) return -1;
        return prioridades[0];
    }
}
```

```
public class Main {

    public static void main(String[] args) {

        ColaPrioridad cola = new ColaPrioridad(10);

        cola.acolarPrioridad(21, 5);
        cola.acolarPrioridad(15, 6);
        cola.acolarPrioridad(7, 8);
        cola.acolarPrioridad(8, 7);
        cola.acolarPrioridad(16, 5);

        System.out.println("Elemento al frente: " + cola.primer());
        System.out.println("Prioridad del frente: " + cola.prioridad());

        System.out.println("\nDesacolar en orden de prioridad:");

        while (!cola.isEmpty()) {
            System.out.println(
                "Elemento: " + cola.primer() +
                " | Prioridad: " + cola.prioridad()
            );
            cola.desacolar();
        }
    }
}
```


IMPLEMENTACIÓN – Estrategia 2



`enqueuePriority(x, p) → void`

Pre: la cola no está llena

`dequeue() → elemento`

Pre: la cola no está vacía

`front() → elemento`

Pre: la cola no está vacía

`priority() → entero`

Pre: la cola no está vacía

`isEmpty() → boolean`

`isFull() → boolean`

elementos →

8	95	7	15	21	8	3	7						
1	3	3	4	5	7	8	9						
0	1	2	3	4	5	6	7	8	9	10	11		

indice →

Elemento

- dato
- prioridad

elementos → 16

prioridades → 5

Elemento

IMPLEMENTACIÓN – Estrategia 2



```
class NodoPrioridad {
    int dato;
    int prioridad;

    public NodoPrioridad(int dato, int prioridad) {
        this.dato = dato;
        this.prioridad = prioridad;
    }
}
```

```
public class ColaPrioridadObj {

    private NodoPrioridad[] cola;
    private int cantidad;
    private int capacidad;

    public ColaPrioridadObj(int capacidad) {
        this.capacidad = capacidad;
        cola = new NodoPrioridad[capacidad];
        cantidad = 0;
    }

    public boolean isEmpty() {
        return cantidad == 0;
    }

    public boolean isFull() {
        return cantidad == capacidad;
    }
}
```

```
public void acolarPrioridad(int x, int p) {
    if (isFull()) return;

    int i = cantidad - 1;

    // desplazar mientras la prioridad sea menor
    while (i >= 0 && cola[i].prioridad < p) {
        cola[i + 1] = cola[i];
        i--;
    }

    cola[i + 1] = new NodoPrioridad(x, p);
    cantidad++;
}

public int desacolar() {
    if (isEmpty()) return -1;

    int valor = cola[0].dato;

    for (int i = 0; i < cantidad - 1; i++) {
        cola[i] = cola[i + 1];
    }

    cantidad--;
    return valor;
}

public int primero() {
    if (isEmpty()) return -1;
    return cola[0].dato;
}

public int prioridad() {
    if (isEmpty()) return -1;
    return cola[0].prioridad;
}
}
```

```
public class Main {

    public static void main(String[] args) {

        ColaPrioridadObj cola = new ColaPrioridadObj(10);

        cola.acolarPrioridad(8, 1);
        cola.acolarPrioridad(95, 3);
        cola.acolarPrioridad(7, 3);
        cola.acolarPrioridad(15, 4);
        cola.acolarPrioridad(21, 5);
        cola.acolarPrioridad(8, 7);
        cola.acolarPrioridad(8, 8);
        cola.acolarPrioridad(7, 8);

        // insertamos el que querías probar
        cola.acolarPrioridad(16, 5);

        System.out.println("Orden de salida por prioridad:\n");

        while (!cola.isEmpty()) {
            System.out.println(
                "Elemento: " + cola.primer() +
                " | Prioridad: " + cola.prioridad()
            );
            cola.desacolar();
        }
    }
}
```



IMPLEMENTACIÓN – Estrategia 3



enqueuePriority

enqueuePriority(x, p) → void

Pre: la cola no está llena

dequeue() → elemento

Pre: la cola no está vacía

front() → elemento

Pre: la cola no está vacía

priority() → entero

Pre: la cola no está vacía

isEmpty() → boolean

isFull() → boolean

1	2	3	4	5
8	95	15	8	21
	7		3	13
			12	

elementos

16

prioridades

5

dequeue


```

public class ColaPrioridad3Colas {
    private Cola colaAlta;
    private Cola colaMedia;
    private Cola colaBaja;

    public ColaPrioridad3Colas(int capacidadPorCola) {
        colaAlta = new Cola(capacidadPorCola);
        colaMedia = new Cola(capacidadPorCola);
        colaBaja = new Cola(capacidadPorCola);
    }

    public boolean isEmpty() {
        return colaAlta.isEmpty() && colaMedia.isEmpty() && colaBaja.isEmpty();
    }

    public boolean isFull() {
        return colaAlta.isFull() || colaMedia.isFull() || colaBaja.isFull();
    }

    public void enqueuePriority(int x, int prioridad) {
        switch (prioridad) {
            case 3:
                colaAlta.enqueue(x);
                break;
            case 2:
                colaMedia.enqueue(x);
                break;
            case 1:
                colaBaja.enqueue(x);
                break;
            default:
                System.out.println("Prioridad inválida");
        }
    }
}

```

- 3 = alta
- 2 = media
- 1 = baja

IMPLEMENTACIÓN – Estrategia 3



```

public int dequeue() {
    if (!colaAlta.isEmpty()) {
        return colaAlta.dequeue();
    } else if (!colaMedia.isEmpty()) {
        return colaMedia.dequeue();
    } else if (!colaBaja.isEmpty()) {
        return colaBaja.dequeue();
    }
    return -1;
}

public int front() {
    if (!colaAlta.isEmpty()) {
        return colaAlta.front();
    } else if (!colaMedia.isEmpty()) {
        return colaMedia.front();
    } else if (!colaBaja.isEmpty()) {
        return colaBaja.front();
    }
    return -1;
}

public int priority() {
    if (!colaAlta.isEmpty()) {
        return 3;
    } else if (!colaMedia.isEmpty()) {
        return 2;
    } else if (!colaBaja.isEmpty()) {
        return 1;
    }
    return -1;
}

```

```

public class Main {
    public static void main(String[] args) {
        ColaPrioridad3Colas cola = new ColaPrioridad3Colas(10);

        cola.enqueuePriority(101, 1); // baja
        cola.enqueuePriority(202, 3); // alta
        cola.enqueuePriority(303, 2); // media
        cola.enqueuePriority(404, 3); // alta
        cola.enqueuePriority(505, 1); // baja
        cola.enqueuePriority(606, 2); // media

        System.out.println("Orden de atención:");

        while (!cola.isEmpty()) {
            System.out.println("Elemento: " + cola.front() +
                               " | Prioridad: " + cola.priority());
            cola.dequeue();
        }
    }
}

```

TDA CONJUNTO (SET)



El conjunto es una estructura que nos permite guardar elementos sin que los mismos se repitan y en donde no se tiene un orden, y además nos permite conocer si un elemento dado se encuentra o pertenece a la estructura.

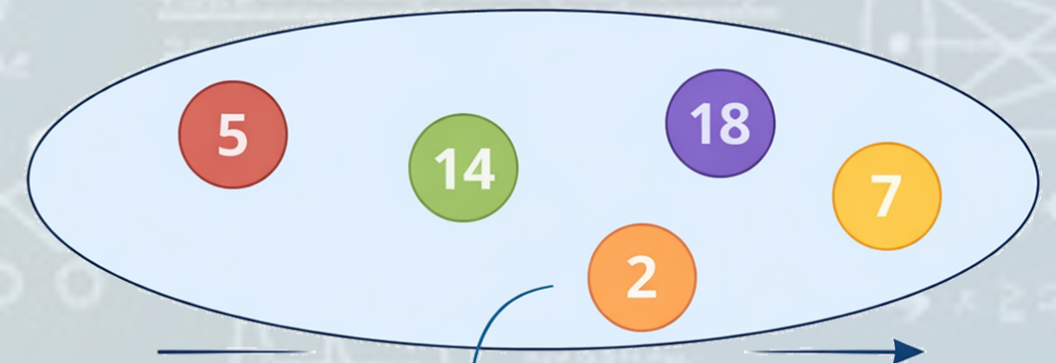
OPERACIONES BÁSICAS

- `add(x)`: agregar elemento si no está
- `remove(x)`: eliminar elemento si está
- `contains(x)`: verificar si pertenece
- `isEmpty()`: verificar si el conjunto está vacío
- `isFull()`: verificar si está lleno
- `size()`: cantidad de elementos
- `clear()`: vaciar el conjunto

PRINCIPIO

- Unicidad sin orden

no se permiten duplicados, no existe orden entre los elementos e importa la pertenencia, no la posición



Sin duplicados

Pertenece?

Sin orden

Operaciones:

- insertar,
- eliminar,
- pertenece

CONJUNTO

El contrato del TDA determina los **estándares** que constituyen la **representación interna** de la estructura.

REPRESENTACION INTERNA



OPERACIONES BÁSICAS

- ✓ **add(x)**: agregar elemento si no está
- ✓ **remove(x)**: eliminar elemento si está
- ✓ **contains(x)**: verificar si el conjunto contiene x
- ✓ **isEmpty()**: es vacío
- ✓ **isFull()**: está lleno
- ✓ **size()**: cantidad de elementos
- ✓ **clear()**: vaciar el conjunto

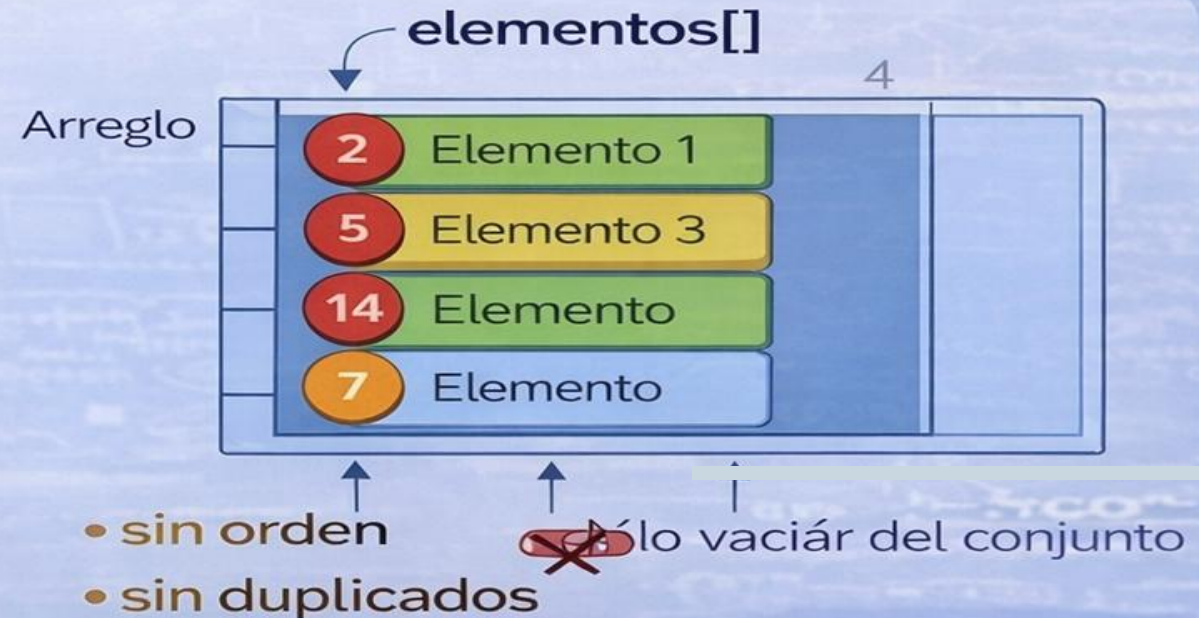
Principio que lo define:

- **Elementos únicos sin orden**
Importa la pertenencia, no la posición



REPRESENTACIÓN INTERNA

¡sin duplicados!



Principio que lo define: **Elementos únicos sin orden**
Importa la pertenencia, no la posición

ESTRATEGIAS DE IMPLEMENTACIÓN



Representación Interna de Conjunto

add(x) → void

Pre: el conjunto no está lleno

x: no pertenece al conjunto

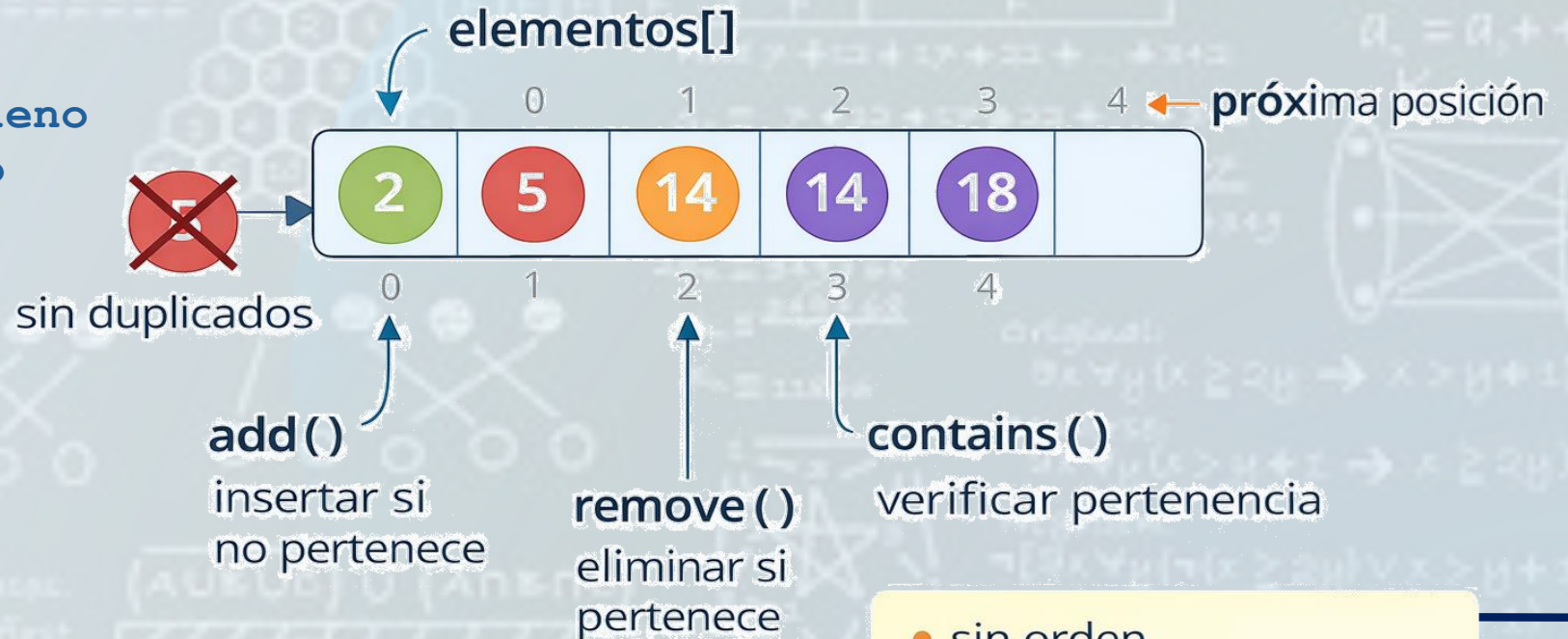
remove(x) → void

contains(x) → boolean

Empty() → boolean

isFull() → boolean

clear() → void



- sin orden
- sin duplicados
- búsqueda por valor

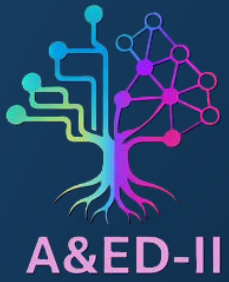
IMPLEMENTACIÓN



```
public class Conjunto {  
  
    private int[] elementos;  
    private int cantidad;  
    private int capacidad;  
  
    public Conjunto(int capacidad) {  
        this.capacidad = capacidad;  
        elementos = new int[capacidad];  
        cantidad = 0;  
    }  
  
    public boolean isEmpty() {  
        return cantidad == 0;  
    }  
  
    public boolean isFull() {  
        return cantidad == capacidad;  
    }  
  
    public boolean contains(int x) {  
        for (int i = 0; i < cantidad; i++) {  
            if (elementos[i] == x) {  
                return true;  
            }  
        }  
        return false;  
    }  
}
```

```
// add  
public void add(int x) {  
    if (!isFull() && !contains(x)) {  
        elementos[cantidad] = x;  
        cantidad++;  
    }  
}  
  
// remove  
public void remove(int x) {  
    for (int i = 0; i < cantidad; i++) {  
        if (elementos[i] == x) {  
            // reemplazar por el último  
            elementos[i] = elementos[cantidad - 1];  
            cantidad--;  
            return;  
        }  
    }  
}  
  
// elegir  
public int elegir() {  
    if (!isEmpty()) {  
        return elementos[0];  
    }  
    return -1;  
}  
  
public int size() {  
    return cantidad;  
}  
}
```

```
public class Main {  
    public static void main(String[] args) {  
  
        Conjunto c = new Conjunto(10);  
  
        c.add(5);  
        c.add(8);  
        c.add(3);  
        c.add(8); // no se agrega (duplicado)  
  
        System.out.println("Elemento elegido: " + c.elegir());  
  
        c.remove(8);  
  
        System.out.println("Después de eliminar 8:");  
        while (!c.isEmpty()) {  
            int x = c.elegir();  
            System.out.println(x);  
            c.remove(x);  
        }  
    }  
}
```



Actividad

Pruebas de TDA Cola con Prioridad y Conjunto

Desarrollar la implementación en JAVA de estos TDA y enviar el código y los resultados obtenidos

Identificadores



- Los identificadores nombran todo aquello que el programador necesite identificar (en Java todo tiene nombre).
- Un identificador comienza con una letra, un guion subrayado (_) o un símbolo de dólar (\$).
- Los siguientes caracteres pueden ser letras o dígitos.
- No se aceptan espacios en el identificador
- Distingue mayúsculas de minúsculas.
- No tienen longitud máxima.



```
edad  
nombreAlumno  
_total  
$precio
```

```
int edadAlumno = 20;
```



```
int e = 20;
```



```
2edad  
nombre alumno  
class
```



Los identificadores son los nombres que el programador utiliza para distinguir los elementos del código.

- Modificadores de alcance.
- Definen desde qué clases se puede acceder a un determinado elemento de un a clase. Existen 4 tipos.
- No poner un modificador explícitamente implica que se utilizará el denominado package_friendly

Modificadores



```
public class Alumno {
    private String nombre;
}
```

	Misma Clase	Mismo Paquete	Subclase	Cualquier Clase
Public	X	X	X	X
Protected	X		X	
Package	X	X		
Private	X			



Los modificadores indican el nivel de acceso o ciertas características del elemento que acompañan.

TIPO DE DATOS PRIMITIVOS



Dato	Tipo	Bits	Rango
Carácter	char	16	0 a 65535
Entero	byte	8	-128 a 127
	short	16	-32768 a 32767
	int	32	-2147483648 a 2147483647
	long	64	-9223372036854775808 a 223372036854775807
Real	float	32	-3.4×10^{38} a -1.4×10^{-45} , 1.4×10^{-45} a 3.4×10^{38}
	double	64	-1.7×10^{308} a -4.9×10^{-324} , 4.9×10^{-324} a 1.7×10^{308}
Booleano	boolean	1	true, false

Para un conjunto de caracteres Java tiene el tipo referencia **String**



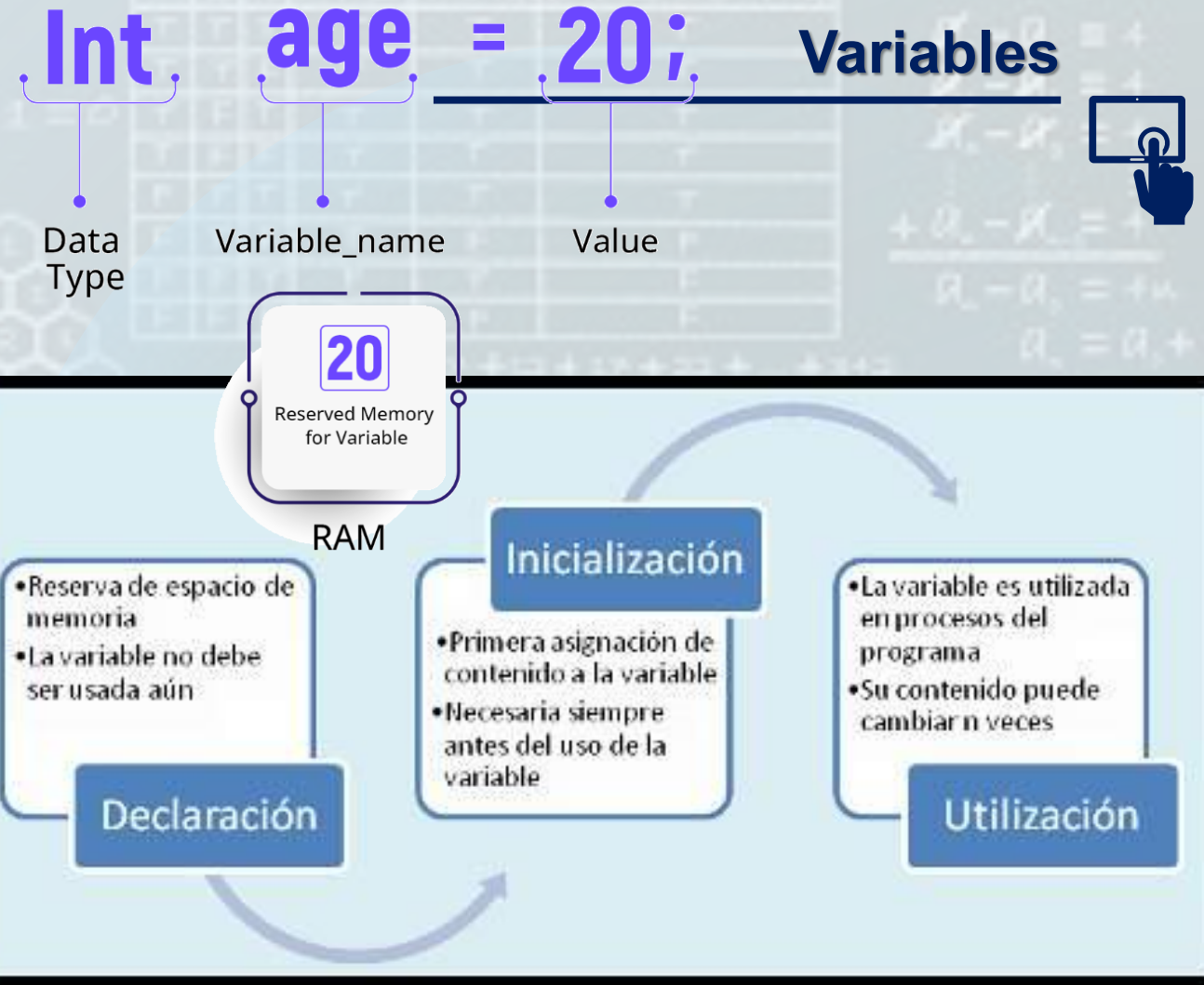
- Java es un lenguaje fuertemente tipado esto significa que toda variable tiene un tipo de datos y es inmutable.

- Variable: nombre simbólico de una posición de memoria.

- [modificador/s] tipo_dato identificador [= expresión / valor] ;

- El tipo me permite verificar que el dato que voy a recuperar de la memoria corresponde al tipo definido.

- Se suele utilizar el estilo lowerCamelCase en el identificador (intercalación de mayúsculas en las palabras en minúsculas).



Una variable es un espacio de memoria identificado por un nombre, destinado a almacenar un valor.

Ejemplos de variable:

```
int edad = 24;
```

//variable declarada e inicializada con un valor.

```
boolean alumnoAceptado = true;
```

```
double salario;
```

//variable declarada pero no inicializada.

```
Automovil autoParticular = new Automovil();
```

//No primitiva definida y creada (inicializada)

```
String nombreDelAlumno;
```

//No primitiva definida sin inicializar

Variables



```
int edad = 25;  
double sueldo = 1500.50;  
boolean activo = true;  
String nombre = "Laura";
```

```
public class DatosAlumno {  
    public static void main(String[] args) {  
        String nombre = "Marcos";  
        int edad = 21;  
  
        System.out.println("Nombre: " + nombre);  
        System.out.println("Edad: " + edad);  
    }  
}
```

**Note que todas las instrucciones
en Java finalizan con un ";"**

*Una variable es un espacio de memoria identificado por un
nombre, destinado a almacenar un valor.*





Tipos de variables según su ámbito

- **Variables de instancia:** Se utilizan para definir los atributos de un objeto.
- **Variables de clase:** Son similares a las variables de instancia, con la excepción de que sus valores son los mismos para todas las instancias de la clase.
- **Variable locales:** Se declaran y se utilizan dentro de las definiciones de los métodos.

Reglas de alcance

```
class MiClase{
```

```
...  
public static void metodo1(...) {
```

```
    ...//bloque  
}
```

```
public void metodo2(...) {  
    ...  
}
```

→ variables de clase

→ parámetros

→ variables locales

→ var. de instancia
→ parámetros
→ variables locales

- Similar a un a variable pero inmutable
- [modificador/s] final tipo_dato
identificador [= expresión / valor] ;
- Los identificadores de constantes se suelen escribir en mayúsculas separándolas por medio de guiones bajos.

Constantes



```
final double PI = 3.1416;  
final int EDAD_MINIMA = 18;
```

```
public class Acceso {  
    public static void main(String[] args) {  
        final int EDAD_MINIMA = 18;  
        int edadUsuario = 20;  
  
        System.out.println("Edad mínima requerida: " + EDAD_MINIMA);  
        System.out.println("Edad del usuario: " + edadUsuario);  
    }  
}
```



Una constante es un valor que, una vez asignado, no puede modificarse.

Operadores aritméticos

+	Suma	$a + b$
-	Resta	$a - b$
*	Multiplicación	$a * b$
/	División	a / b
%	Módulo	$a \% b$

Operadores de comparación

==	Igual	$a == b$
!=	Distinto	$a != b$
<	Menor que	$a < b$
>	Mayor que	$a > b$
<=	Menor o igual que	$a <= b$
>=	Mayor o igual que	$a >= b$

Operadores



Java™

Operadores de asignación

=	Asignación	$a = b$
+=	Suma y asignación	$a += b$ ($a = a + b$)
-=	Resta y asignación	$a -= b$ ($a = a - b$)
*=	Multiplicación y asignación	$a *= b$ ($a = a * b$)
/=	División y asignación	$a /= b$ ($a = a / b$)
%=	Módulo y asignación	$a \% = b$ ($a = a \% b$)



() - *paréntesis*

Lista de parámetros - Precedencia en expresiones
Conversiones de tipo de valores.

{ } - *llaves*

Definir un bloque de código - Inicialización de matrices.

[] - *corchetes*

Declaración de arreglos - Referencia.

, - *coma*

Separador de elementos en listas de valores.

; - *punto y coma*

Separa sentencias.

. - *punto*

Separador de espacios de nombres.



Comentarios



// - Comentarios para una sola línea

```
// Comentario de una sola línea
```

/*
comentarios de una o más líneas
de texto.

```
/* Este es un comentario  
tradicional. Puede  
dividirse en muchas líneas */
```

*/

/**
* Comentario de documentación, Entrada para el
* javadoc.exe
*/

```
/**  
* Programa para imprimir texto.  
* @author ProfesorJava  
* @version 2010  
*/
```

```
public class Comentarios {  
    public static void main(String[] args) {  
        // Mostrar un mensaje de bienvenida  
        System.out.println("Hola");  
    }  
}
```


Condicionales



Se utilizan para alterar la secuencia de ejecución.

```
public class Aprobacion {  
    public static void main(String[] args) {  
        int nota = 8;  
  
        if (nota >= 6) {  
            System.out.println("Aprobado");  
        }  
    }  
}
```

```
public class Aprobacion {  
    public static void main(String[] args) {  
        int nota = 4;  
  
        if (nota >= 6) {  
            System.out.println("Aprobado");  
        } else {  
            System.out.println("Desaprobado");  
        }  
    }  
}
```

If - else

If(expression_booleana)

{

sentencias;

}

else

{

sentencias;

}





Switch

```
switch( expr1 )
{
    case valor1:
        sentencias;
        break;
    case valor2:
        sentencias;
        break;
    [default:
        sentencias;]
}
```

```
public class DiaSemana {
    public static void main(String[] args) {
        int dia = 2;

        switch (dia) {
            case 1:
                System.out.println("Lunes");
                break;
            case 2:
                System.out.println("Martes");
                break;
            default:
                System.out.println("Otro día");
        }
    }
}
```



- Compara una constante con el valor de la expresión.
- Si lo encuentra, ejecuta las sentencias asociadas hasta el break.
- Si no lo encuentra busca la línea default y ejecuta las sentencias asociadas.
- La sentencia default es opcional.

Repiten un bloque de sentencias un numero fijo de veces o hasta que se cumpla una condición.

For

```
for( expr1 inicio; expr2 test; expr3 incremento)
{
    sentencias;
}
```

Ejemplo:

```
for (int i = 0; i<10; i++)
{
    sentencias;
}
```

```
public class Repeticion {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Valor: " + i);
        }
    }
}
```





While:

```
while( condición booleana ) {  
    sentencias;  
}
```

Ejemplo:

```
while(condición booleana ) {  
    sentencias;  
}
```

El conjunto de sentencias encerrada entre las llaves se ejecuta mientras la condición se evalúe como verdadera.

```
public class RepeticionWhile {  
    public static void main(String[] args) {  
        int i = 1;  
  
        while (i <= 3) {  
            System.out.println(i);  
            i++;  
        }  
    }  
}
```





Do/Wile:

```
do {  
    sentencias;  
}while( condición booleana );
```

Ejemplo

```
do {  
    sentencias;  
}while(condición booleana);
```

Similar al while, pero por lo menos una vez se ejecuta sin importar el valor de la expresión booleana.



```
public class RepeticionDoWhile {  
    public static void main(String[] args) {  
        int i = 1;  
  
        do {  
            System.out.println(i);  
            i++;  
        } while (i <= 3);  
    }  
}
```

Interrupción del flujo de ejecución



break: Rompe la iteración de los bucles while, for y do-while.

continue: Rompe una iteración de un bucle provocando que se ejecute la siguiente iteración ignorando las sentencias posteriores a la sentencia continue.

return: termina la ejecución de una función y devuelve el valor indicado. Si la misma se halla en una estructura de iteración, también esta termina, aunque no halla cumplido su condición.



Mecanismos de Entrada y Salida



System.out.print / System.out.println

- Representación de datos en la consola.
- Cast de cualquier cosa a carácter.
- Posee varias **“secuencias de escape”** para formatear la salida.

\b (borra hacia atrás) \n (nueva línea)
\t (tabulador) \r (salto de línea)
\\ (barra invertida) \" (comilla doble)
\` (comilla simple)



System.out.println

Inserts a new line

System.out.print

Keeps printing to the same line

Mecanismos de Entrada y Salida

System.in.read():

- Mecanismo básico de lectura en Java.
- Obtiene la representación ASCII del carácter.
- Necesita controlar las excepciones.

InputStreamReader y BufferedReader:

- Permiten (en conjunto) facilitar la lectura de conjuntos de caracteres.
- Lo obtenido, si no es una cadena de caracteres debe ser convertido al tipo correspondiente.

Scanner (a partir de versión 1.5):

- Permite leer conjuntos de caracteres y formatearlos en un solo paso.
- No necesita control de excepciones.

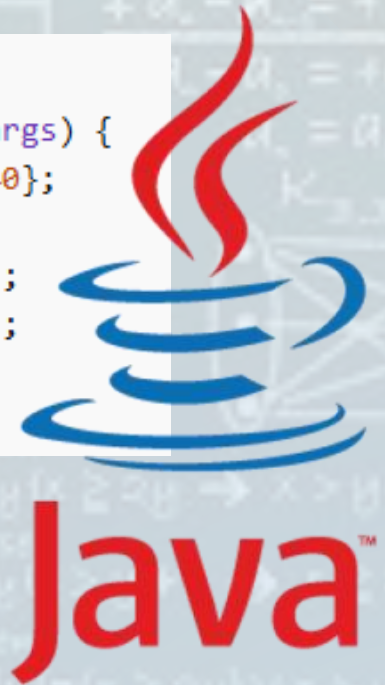


Arreglos



- Secuencia de posiciones de memoria, todas bajo el mismo nombre y del mismo tipo, se accede por su posición ordinal en la estructura.
- Pueden ser de primitivos o de objetos.
- Se pueden crear arreglos vacíos o con valores iniciales.
- No se pueden crear arreglos en tiempo de compilación.
- No se puede utilizar un arreglo sin declarar el tamaño.
- **TODOS** los arreglo en Java son **ESTÁTICOS**.

```
public class ArregloSimple {  
    public static void main(String[] args) {  
        int[] numeros = {10, 20, 30, 40};  
  
        System.out.println(numeros[0]);  
        System.out.println(numeros[2]);  
    }  
}
```



Todos los elementos de un arreglo deben ser del mismo tipo. Además, su tamaño se define al momento de crearlo.

Ejemplos:

Definir un arreglo de enteros de 4 elementos sin inicializar.

```
int valores[] = new int[4];
```

Definir un arreglo de floats de 4 elementos inicializado.

```
float valorHora[] = {15.50f,18.10f,20.00f,10.30f}
```

Definir un arreglo bidimensional (matriz) de enteros de 10 elementos por imensión sin inicializar.

```
int[][] matriz = new int[10][10]
```

○ Definir un arreglo de enteros de 2 por 3 inicializando.

○

```
int[][] matricita = {{1,2,3},{4,5,6}}
```





- Conjunto de operaciones con un objetivo determinado asociadas a un nombre simbólico
- [modificador/s] tipo_retorno
identificador([lista_de_parámetros]) {{
| ;}}
- Utilizan en su identificador el estilo lowerCamelCase
- Existen distintos tipos de métodos dependiendo el objetivo

Ejemplos.

float calcularSalario(int nroEmpleado){...}

String obtenerNombre();

```
public class Saludo {  
  
    public static void saludar() {  
        System.out.println("Bienvenido al sistema");  
    }  
  
    public static void main(String[] args) {  
        saludar();  
    }  
}
```

```
public class Operacion {  
  
    public static int sumar(int a, int b) {  
        return a + b;  
    }  
  
    public static void main(String[] args) {  
        int resultado = sumar(3, 4);  
        System.out.println("Resultado: " + resultado);  
    }  
}
```



Un método es un bloque de instrucciones con un nombre y un objetivo específico. Los métodos permiten organizar el programa y evitar repeticiones.

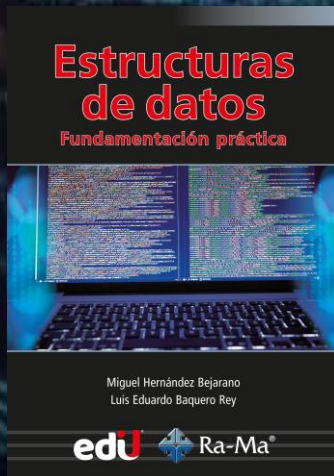
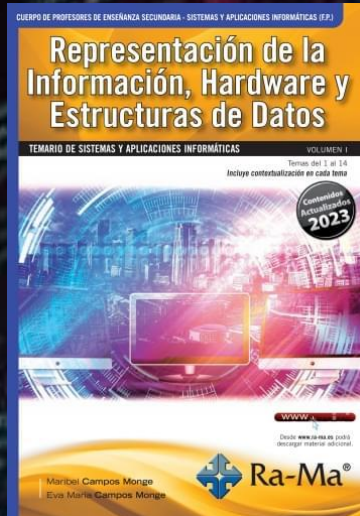




Actividad

Creación de código





BIBLIOGRAFIA



Cuadrado Estrebou, María Fernanda Trutner, Guillermo Hernán. Revisión: J.Tempio.
Programación II. Apuntes de cátedra.



PROXIMO ENCUENTRO



Clase 4

Diccionario: clave, valor y diseño modular

TDA Diccionario (simple y múltiple).

Interfaces e implementación en Java.

Separación entre contrato y lógica interna.